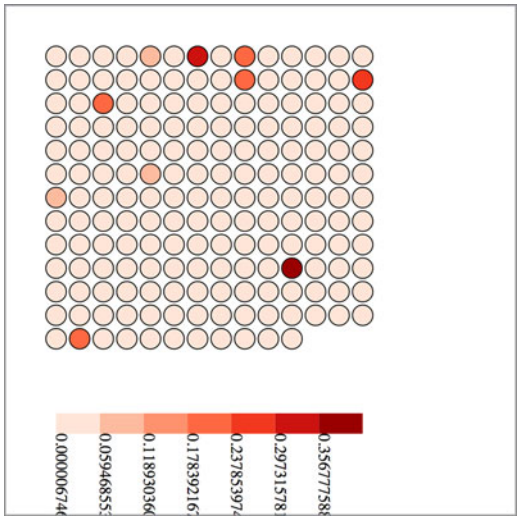


Figure 10.11 Our legend with rudimentary labels

Listing 10.15 Text labels for legend

```
gSelection.selectAll("text")
   .data(data)
   .enter()
   .append("g")
   .attr("transform", function (d) {
      return "translate(" + xScale(d.domain[0]) +"," + size[1] + ")";
   })
   .append("text")
      .attr("transform", "rotate(90)")
      .text(function(d) {return d.domain[0];});
```

The text element needs to be placed in a g so that it can be translated and then rotated; otherwise, it'll be rotated and then translated, which would place it at the translation relative to its new rotation (taking the text off the page).

As shown in figure 10.11, they aren't the prettiest labels. We could adjust their positioning, font, and style to make them more effective. They also need functions like the grid layout has to define size or other elements of the component.

This is usually the point where I say that the purpose of this chapter is to show you the structure of components and layouts, and that making the most effective layout or component is a long and involved process that we won't get into. But this is an ugly legend. The break points are hard to read, and it's missing pieces that the component needs, such as a title and an explanation of units.

10.2.3 Adding component labels

Let's add those features to the legend, and create ways to access them, as shown in listing 10.16. We're using `d3.format`, which allows us to set a number-formatting rule based on the popular Python number-formatting mini-language (found at https://docs.python.org/release/3.1.3/library/string.html#formatspec).